CASE STUDY · SELF-INITIATED PROJECT

From an Empty VM to a Campus Lab Platform: Building NIBM's Student Lab from Scratch

Supun Hewagamage · Technical Assistant, National Institute of Business Management (NIBM), Sri Lanka

github.com/supunhg · November 2026

At a glance	
Role	Technical Assistant, while studying for a degree in Ethical Hacking and Network Security. Self-initiated: conceived, designed, built and operated the entire platform alone
Starting point	No student lab server existed at NIBM. Built from a bare VM on the campus ESXi host
Environment	Dell PowerEdge on VMware ESXi → one Ubuntu 24.04 VM (8 vCPU, 16 GB RAM), Synology NAS, campus LAN behind a Sophos firewall
Scope	34 containers · 24 HTTPS services · students, staff and administrators
Core stack	Docker Compose, Authentik, Nginx Proxy Manager (OpenResty + Lua), AdGuard Home, ZeroTier, Qdrant, Forgejo, Flarum, Paperless-ngx, Vaultwarden, n8n, Uptime Kuma, Beszel, ntfy, restic
Outcome	One identity per person across every service, self-service onboarding with admin approval, automatic offboarding, monitored and backed-up, fully documented for handover

1. Background: building it, then making it safe

When I joined NIBM as a Technical Assistant there was no lab server for students. On my own initiative I created one: an Ubuntu VM on the campus VMware ESXi host with internal DNS, a reverse proxy, remote access and around twenty self-hosted services: a student forum, a password manager, workflow automation, a vector database for AI projects and more.

I was learning as I built, so version 1 focused on getting useful services running. When I audited my own work against what an institution needs, the gaps were clear:

- The vector database I had deployed had **no authentication**: anyone on the network could read or delete every collection.
- **No host firewall**: every port was reachable from the LAN, including the Docker management console (equivalent to root on the server).
- **No backups** and no alerting: a storage mount failed silently for two weeks before I noticed.
- **Default credentials** remained on one admin panel, and every application kept its own, unrelated list of users.

Phase two was to mature my own build into a platform the institute could rely on after my contract ends: secure by default, simple for students, and manageable by one administrator through a browser.

2. Architecture

Every request now passes through one front door. AdGuard Home resolves the internal *.lab names; a hardened reverse proxy terminates HTTPS for 24 services; Authentik decides who may enter; and group membership drives everything behind it.

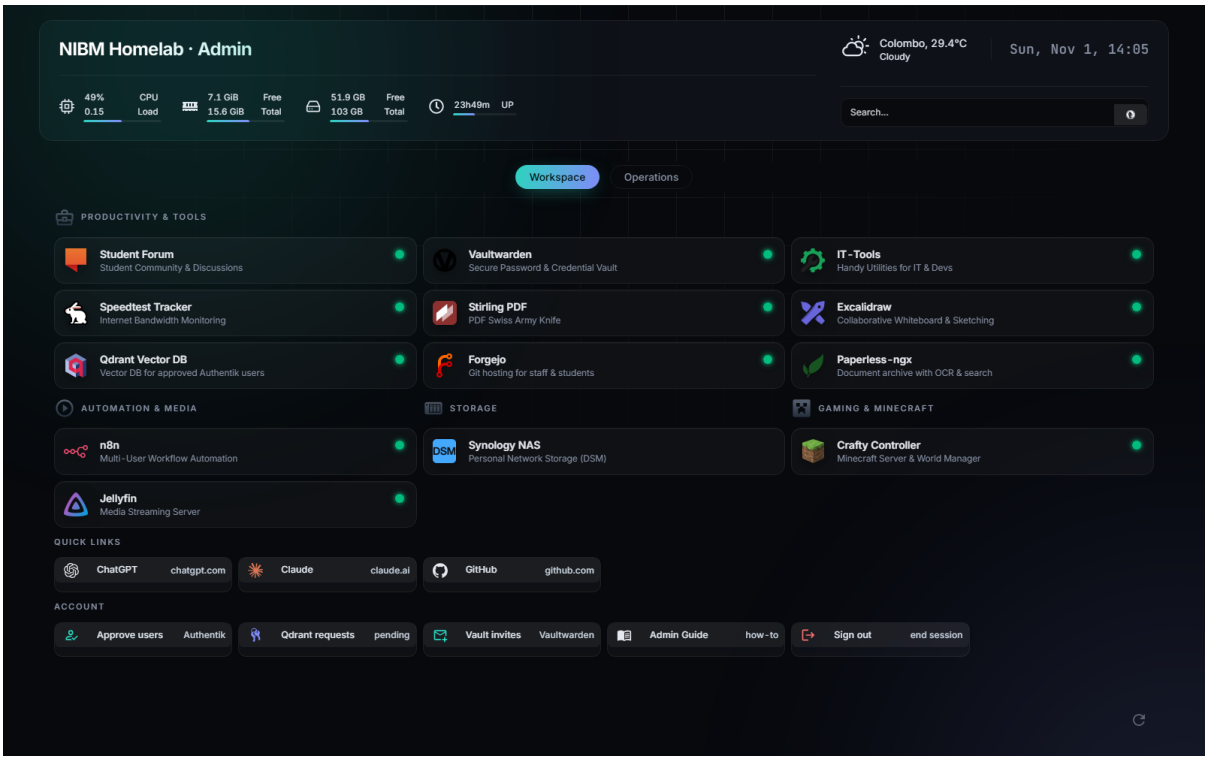


Figure 1: Admin dashboard at home.lab. Students signing in at the same URL see a separate student hub.

Layer	Design
Identity	Authentik with infrastructure-as-code blueprints: approval-gated self-registration, groups, forward-auth applications, custom expression policies and scope mappings
Edge	Nginx Proxy Manager / OpenResty, all hosts generated from one script (HTTPS redirect, HSTS); Lua for access decisions that must run after authentication
Applications	Docker Compose with pinned images, data under /opt/homelab/<service>, secrets isolated in a mode-600 .env
Network	iptables policy that also filters Docker-published ports (DOCKER-USER chain), ZeroTier for remote administration, fail2ban, key-only SSH
Operations	Uptime Kuma (25 probes), Beszel metrics, Dozzle logs, Diun update checks, ntfy push alerts, nightly restic backups to the NAS

3. Key engineering decisions

3.1 One identity per person

Students self-register with their NIBM index number as the username (format enforced by an expression policy, e.g. cohndne251f-019). Emails and usernames must be unique, and new accounts stay inactive until an administrator approves them; the administrator gets a push notification. From then on, group membership controls every service.

3.2 Role-based dashboards on a single URL

home.lab runs two dashboard instances. After authentication, an nginx map on the identity provider's groups header routes administrators to the full cockpit and students to a curated hub. Using a map rather than if/set was essential: rewrite-phase directives execute before the authentication sub-request populates the header.

3.3 Scoped, expiring keys for the vector database

Rather than handing users the database master key, a custom Authentik scope mapping mints a signed JWT at each login, encoding exactly the user's tier: read-only, read/write on named collections, or full management. The proxy injects it as the API key, and a self-service page lets students copy their own key for code. Access requests are a short form that records the purpose, adds the user to a pending group and alerts the administrator.

> **Verified, not assumed**
>
> Every tier is covered by an end-to-end test that performs real logins and checks Qdrant's own responses: readers get 403 on writes, per-collection users get 403 outside their collection, managers can create collections.

3.4 Identity-bound application accounts

The forum and Git server keep their own accounts, but a Lua guard on their sign-up endpoints only accepts registrations whose username and email match the signed-in lab identity. A small sync service then confirms matching accounts automatically and suspends a student's Git login within two minutes of their lab account being disabled, so offboarding happens in one place.

3.5 Everything as code, everything tested

Compose files, identity blueprints, proxy generation, firewall rules and systemd units live in version control. Three end-to-end suites (dashboard routing, vector-database tiers, identity binding) exercise the real login flows after every change.

4. Security hardening

#	Finding	Resolution
1	Vector database unauthenticated	API key + JWT RBAC; dashboard behind SSO; per-user scoped keys

#	Finding	Resolution
2	No host firewall; Docker ports bypass UFW-style rules	iptables policy incl. DOCKER-USER filtering by original destination port
3	Factory admin password on a monitoring app	Rotated; consoles placed behind an SSO gate (two locks)
4	Password SSH, root login, no brute-force protection	Key-only SSH, AllowUsers, fail2ban
5	No backups	Nightly encrypted restic to NAS, 7/4/6 retention, weekly integrity check, restore tested
6	Duplicate emails possible at sign-up	Unique username/email policy (prevents account takeover in email-linked apps)
7	Unpinned :latest images	All images pinned; Diun pushes new versions to the admin
8	Secrets in plaintext compose file	Moved to a mode-600 .env; generated where new

22 findings in total are tracked in the project's security log, with severity, fix and status.

5. Operations and reliability

- 25 uptime probes, each wired to push notifications; a public status page for students.
- Per-container CPU/RAM/disk history and live, searchable logs for the administrator only.
- Nightly backups include consistent database dumps, application data, a vector-database snapshot, firewall/SSH configuration and the ZeroTier identity.
- Host tuning (BBR congestion control, buffer sizing, swappiness) and per-service limits reduced RAM in use by 22% while adding 13 services.

6. Results

Metric	Result
Services	34 containers, 24 HTTPS hosts generated from one script
Identity	1 account per person across dashboard, forum, Git, vector DB and consoles
Onboarding	Self-registration → one-click approval → automatic activation in forum and Git
Offboarding	Disable once; access ends everywhere within 2 minutes
Security	22 findings fixed or tracked, including 3 critical
Efficiency	−22% RAM in use while adding 13 services
Quality	3 end-to-end test suites using real logins

7. Lessons learned

- **Identity first.** Once one system owns "who is this person", every other feature (dashboards, keys, offboarding) becomes a group lookup.
- **Know your request phases.** Several fixes came down to nginx phase ordering: maps and access_by_lua run after authentication; if/set do not.
- **Containers bypass naive firewalls.** Published Docker ports never touch the INPUT chain; filtering belongs in DOCKER-USER, keyed on the original destination port.
- **Test the real path.** Scripted logins caught issues unit checks never would: CSRF behaviour, redirect methods and an outpost that needed a restart after first boot.
- **Write for the successor.** The admin guide covers navigation, daily tasks, credentials, recovery and a handover checklist, so the lab does not depend on one person.

8. Handover and next steps

The platform ships with an Admin Guide, a Student Guide and a from-scratch Setup Guide. Recommended next steps for the institute: deploy the prepared internal certificate authority (removes browser warnings and enables full single sign-on for the remaining apps), add an off-site backup copy, connect a campus SMTP relay, and move administrative accounts (ZeroTier, Cloudflare) to institutional ownership.

Supun Hewagamage · github.com/supunhg